Presentation Agenda

What We Will Cover Today

01	Introduction to Testing Techniques
02	Categories: Black Box, White Box, Grey Box
03	Black Box Testing Techniques (Full Detail)
04	White Box Testing Techniques (Full Detail)
05	Grey Box Testing Techniques
06	Static vs Dynamic Testing
07	Experience-Based Testing
08	Risk-Based Testing
09	Test Design in Automation (Selenium / Playwright)
10	Real-World Application Scenario
11	Best Practices & Common Mistakes
12	Summary & Q&A

SECTION 01

Introduction to Testing Techniques

Understanding the foundation of structured software quality validation

What Are Software Testing Techniques?

Section 01 — Introduction

Definition

Software Testing Techniques are structured, systematic methods used to design, select, and execute test cases that validate whether a software application meets its requirements and functions correctly in all expected — and unexpected — scenarios.

Structured Approach

Rather than testing randomly, techniques provide a defined strategy for selecting what to test and how to test it — ensuring nothing critical is missed.

Coverage Maximization

Techniques help identify all relevant inputs, states, and conditions to be tested, minimizing gaps in test coverage.

Defect Detection

Each technique targets specific types of defects — boundary errors, logic flaws, state issues — allowing teams to find bugs efficiently.

Industry Alignment

All major QA frameworks (ISTQB, IEEE 829) and automation tools (Selenium, Playwright) are built around these proven techniques.

Why Testing Techniques Are Important

Section 01 — Introduction

Industry Impact

80%

of software defects found in testing never reach production when proper techniques are applied

60%

reduction in test case redundancy when using structured techniques vs random testing

$1.5T

lost globally each year due to poor software quality (Consortium for IT Software Quality)

✓

Prevents Costly Production Bugs

Finding a bug in production costs 10–100× more than finding it during testing. Proper techniques surface defects early.

✓

Improves Test Coverage

Techniques ensure edge cases, boundary values, and complex logic paths are systematically tested — not guessed.

✓

Enables Repeatable, Reliable Testing

Documented techniques make tests reproducible across team members, releases, and environments.

✓

Supports Automation

Tools like Selenium and Playwright rely on the same principles — equivalence partitioning, boundary analysis — to build robust automated test suites.

✓

Satisfies Regulatory Requirements

In industries like banking, healthcare, and aerospace, using documented testing techniques is often a compliance requirement.

SECTION 02

Categories of Testing Techniques

Black Box • White Box • Grey Box

Categories of Testing Techniques

Three Fundamental Perspectives on Software Testing

Black Box Testing

External Behavior

The tester has NO knowledge of the internal code, architecture, or implementation. Testing is based entirely on input/output behavior and requirements.

Primarily used by: QA Engineers, End Users

Key Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing
- Error Guessing

White Box Testing

Internal Structure

The tester has FULL knowledge of the internal code structure, logic, and implementation. Tests are designed based on the source code itself.

Primarily used by: Developers, Senior QA Engineers

Key Techniques:

- Statement Coverage
- Branch Coverage
- Path Coverage
- Loop Testing
- Control Flow Testing

Grey Box Testing

Hybrid Approach

The tester has PARTIAL knowledge of the internal system — typically the architecture and data structures but not the full source code. Combines both approaches.

Primarily used by: QA Engineers with Dev background

Key Techniques:

- Integration Testing
- Penetration Testing
- Database Testing
- Web Service / API Testing

Comparison: Black Box vs White Box vs Grey Box

At-a-Glance Reference Guide

Criteria	Black Box	White Box	Grey Box
Knowledge of Code	None	Full	Partial
Who Performs	QA Testers	Developers / Senior QA	QA Engineers w/ dev skills
Test Basis	Requirements & Specs	Source Code & Logic	Architecture + Specs
Focus Area	External behavior	Internal paths/logic	Integration & security
Strengths	User-perspective, broad coverage	Deep code logic coverage	Balanced, realistic scenarios
Limitations	Misses internal logic gaps	Requires code access	Complexity in hybrid approach
Automation Tools	Selenium, Playwright	JUnit, JaCoCo, Emma	Postman, Burp Suite
ISTQB Category	Specification-based	Structure-based	Experience + Structure

SECTION 03

Black Box Testing

Techniques

Testing from the user's perspective — no internal knowledge required

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing
- Error Guessing

Equivalence Partitioning

Black Box Technique #1 — Definition & Purpose

Definition

Equivalence Partitioning (EP) divides the range of possible input values into groups (partitions or classes) where all values in a group are expected to be processed identically by the software. One test case is then selected from each partition to represent the entire group.

Valid Partition

Inputs that the system SHOULD accept and process correctly. Tests in this class verify that the system handles expected values properly.

EXAMPLE

Age field (18–65): Testing with 35 — valid, expected to pass.

Invalid Partition (Low)

Inputs BELOW the valid range. These should be rejected by the system with an appropriate error message.

EXAMPLE

Age field: Testing with 5 — below minimum, should be rejected.

Invalid Partition (High)

Inputs ABOVE the valid range. These should also trigger validation errors and be handled gracefully.

EXAMPLE

Age field: Testing with 200 — above maximum, should be rejected.

Equivalence Partitioning — Step-by-Step Process

How to Apply This Technique in Practice

1 Identify the Input Field / Parameter

Identify each input field or parameter that needs to be tested. Example: A registration form has fields for Name, Email, Age, and Password.

2 Determine the Valid Range

Read the requirements to understand what values are considered valid. Example: Age must be between 18 and 65 (inclusive).

3 Define the Partitions

Create at least three partitions: one valid class and at least one invalid class on each side (below and above).

4 Select One Representative Value per Partition

Pick any one value from each partition. The key insight: if one value in a partition works/fails, they all should.

5 Design Test Cases

Write one test case per selected value. Document expected results for each test.

6 Execute and Record Results

Run the tests, capture actual results, and compare with expected outcomes. Log defects for any mismatch.

Boundary Value Analysis (BVA)

Black Box Technique #2 — Testing at the Edges

Boundary Value Analysis extends Equivalence Partitioning by focusing specifically on the BOUNDARIES between partitions. Defects are statistically most common at boundary values — off-by-one errors are one of the most frequent bugs in software development.

Example: Age field accepting values 18 to 65

BVA DIAGRAM

Why Boundaries?

Developers commonly make off-by-one mistakes (e.g., using > instead of >= in a condition). BVA specifically targets these error-prone areas.

BVA Values to Test

For each boundary, test: minimum − 1 (invalid), minimum (valid), minimum + 1 (valid), maximum − 1 (valid), maximum (valid), maximum + 1 (invalid).

Real-World Example

A bank's transfer limit is $1 to $10,000. BVA tests: $0, $1, $2, $9,999, $10,000, $10,001 to catch boundary handling errors.

Decision Table Testing

Black Box Technique #3 — Condition-Action Mapping

Decision Table Testing is used when a system's behavior depends on MULTIPLE CONDITIONS acting simultaneously. The table maps all combinations of conditions to their corresponding actions — making it impossible to miss a combination.

Example Scenario: Online Loan Application System

Conditions: Credit Score (Good/Poor) × Employment Status (Employed/Unemployed) × Loan Amount (Low/High)

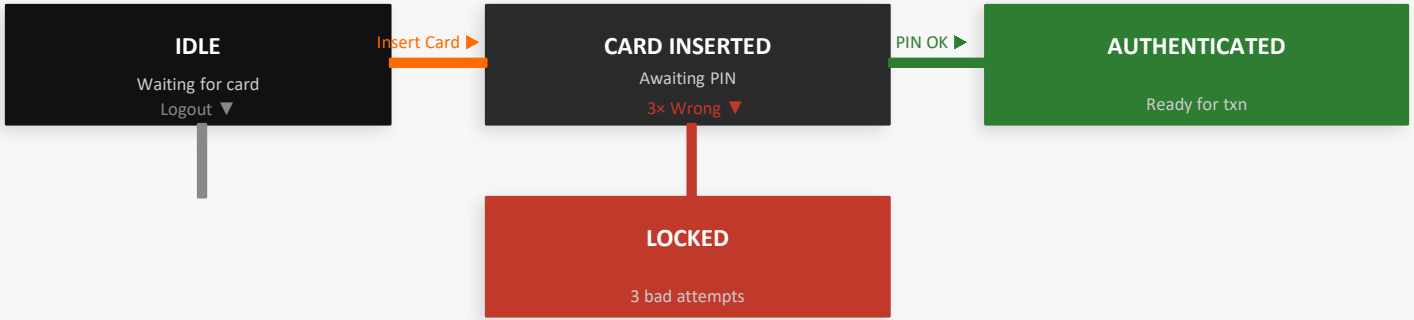
Conditions / Actions	Test 1	Test 2	Test 3	Test 4	Test 5
C1: Credit Score Good?	Y	Y	Y	N	N
C2: Employed?	Y	Y	N	Y	N
C3: Loan Amount Low?	Y	N	Y	Y	N
Action: Approve Loan	✓	✓	✗	✗	✗
Action: Request Review	✗	✗	✓	✓	✗
Action: Reject	✗	✗	✗	✗	✓

State Transition Testing

Black Box Technique #4 — System States & Transitions

State Transition Testing models a system as a finite state machine. States represent conditions; events trigger transitions between them. Test cases validate every state and every valid/invalid transition.

Example: ATM System — State Machine



TC#	Start State	Event / Input	Expected End State
TC1	IDLE	Insert valid card	CARD INSERTED
TC2	CARD INSERTED	Enter correct PIN	AUTHENTICATED
TC3	CARD INSERTED	Wrong PIN × 3	LOCKED
TC4	AUTHENTICATED	Log Out / Session timeout	IDLE

Use Case Testing & Error Guessing

Black Box Techniques #5 and #6

Use Case Testing

Use Case Testing derives test cases from use cases — documented descriptions of how a USER interacts with a system to achieve a GOAL. Each use case maps a main flow and alternative/exception flows.

Example — Login Use Case:

- Main Flow: Valid credentials → Authenticated → Dashboard
- Alt Flow 1: Wrong password → Error shown → User retries
- Alt Flow 2: Forgot Password → Reset email sent
- Exception Flow: System unavailable → Error page shown

Test cases cover EACH flow path — including exception paths developers often overlook.

Error Guessing

Error Guessing is an experience-based technique where testers use knowledge of common mistakes and past defects to intuitively design test cases. No formal structure — driven by expertise.

Common Errors Testers Target:

- Empty inputs / null / blank fields
- Zero and negative values in numeric fields
- Extremely long text strings (buffer overflow)
- Special characters: !@#$%^&*() in name fields
- Duplicate submissions (double-click bug)
- Invalid dates: 31/13/2024, 00/00/0000
- SQL injection strings in input fields
- Network disconnection mid-transaction

Best combined with structured techniques — not used in isolation.

SECTION 04

White Box Testing

Techniques

Code-level testing — validating every path, branch, and statement in the source code

- Statement Coverage
- Branch Coverage
- Path Coverage
- Loop Testing
- Control Flow Testing

Statement Coverage & Branch Coverage

White Box Techniques #1 and #2

Statement Coverage

Goal: Ensure every EXECUTABLE LINE of code is executed at least once.

Formula: Statements Covered ÷ Total Statements × 100%

```
def check_discount(age, member):
  discount = 0              # S1
  if age > 60:              # S2
    discount = 20           # S3
  if member == True:        # S4
    discount += 10          # S5
  return discount           # S6
```

To achieve 100% statement coverage:

- TC1: age=70, member=True → Covers S1–S6
- TC2: age=40, member=False → Covers S1, S2, S4, S6

S3 and S5 only covered when conditions are true.

Branch Coverage

Goal: Ensure both TRUE and FALSE outcomes of every DECISION POINT are executed.

Formula: Branches Covered ÷ Total Branches × 100%

Branch coverage is STRONGER than statement coverage. 100% branch coverage automatically gives 100% statement coverage — but NOT vice versa.

Example Branches to Test:

if age > 60 → TRUE Input: age=70	S3 executes (discount=20)
if age > 60 → FALSE Input: age=40	S3 skipped
if member==True → TRUE Input: member=True	S5 executes (discount+=10)
if member==True → FALSE Input: member=False	S5 skipped

Path Coverage & Loop Testing

White Box Techniques #3 and #4

Path Coverage

Goal: Ensure every POSSIBLE PATH through the code (entry to exit) is executed at least once. Most thorough but computationally expensive.

Key Concept: Cyclomatic Complexity

Cyclomatic Complexity = E − N + 2P
(Edges − Nodes + 2 × Connected components)

This number tells you the MINIMUM test cases needed to achieve full path coverage.

All paths in discount example:

- Path 1: S1→S2(F)→S4(F)→S6 — age≤60, not member
- Path 2: S1→S2(T)→S3→S4(F)→S6 — senior, not member
- Path 3: S1→S2(F)→S4(T)→S5→S6 — not senior, member
- Path 4: S1→S2(T)→S3→S4(T)→S5→S6 — senior + member

Loop Testing

Goal: Specifically test loops (for/while/do-while) — common sources of off-by-one errors, infinite loops, and exit condition bugs.

Simple Loop:

- 0 iterations (skip entirely)
- 1 iteration
- 2 iterations
- Typical middle value
- Max − 1 iterations
- Max iterations
- Max + 1 (should fail/stop)

Nested Loop:

- Min values for all loops
- Inner max, outer min
- Both loops at maximum
- Test each loop independently

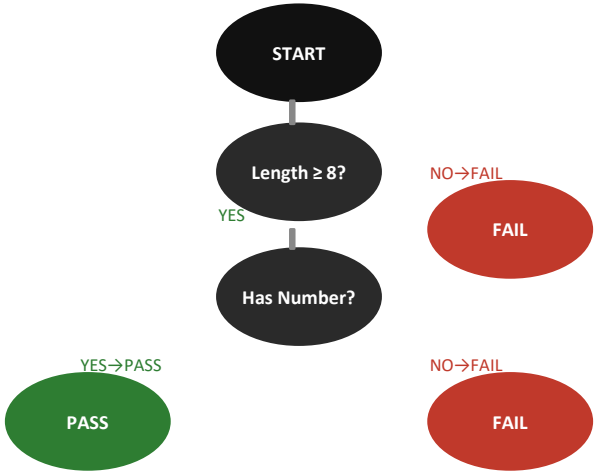
Infinite loop risk: Always test loops where the exit condition might never be reached.

Control Flow Testing

White Box Technique #5 — Graph-Based Execution Analysis

Control Flow Testing uses a Control Flow Graph (CFG) — a visual representation of ALL POSSIBLE EXECUTION PATHS through a program. Each node represents a block of code; edges represent transitions. This enables systematic path identification.

CFG — Password Validator



CFG Components

- Node (Basic Block): Statements with no internal branches
- Edge: Transition from one block to another
- Decision Node: Point with multiple outgoing edges
- Entry Node: Start of execution
- Exit Node: Return / end of function

How to Build a CFG (Steps)

- Step 1: Break code into basic blocks (no branches inside)
- Step 2: Draw each block as a node
- Step 3: Add directed edges for each possible transition
- Step 4: Label edges with conditions (True/False)
- Step 5: Calculate cyclomatic complexity to count paths

Tools Used in Industry

- IntelliJ IDEA: Built-in control flow analyzer
- Eclipse: Control Flow plugin available
- SonarQube: Static code path analysis
- JaCoCo: Java code coverage (branch + statement)

SECTIONS 05 & 06

Grey Box Testing &

Static vs Dynamic Testing

Hybrid approaches and execution vs non-execution based testing

Grey Box Testing Techniques

Section 05 — The Hybrid Approach

Grey Box Testing sits between Black Box and White Box testing. The tester has PARTIAL knowledge — typically the database schema, API contracts, and architecture — but NOT the full source code. This allows more targeted, realistic tests.

API / Web Service Testing

Tester knows the API contract (endpoints, request/response structure) but not the backend implementation. Tests validate correct responses, error codes, and data integrity.

Example: Using Postman: POST /users with valid JSON returns 201 Created and the record appears in the database.

Database Testing

Tester has access to the database schema and can write SQL queries to validate that front-end operations correctly persist data — without seeing the backend code.

*Example: After user registration: SELECT * FROM users WHERE email='test@test.com' — verify all fields stored correctly.*

Integration Testing

Tester knows how modules communicate (interfaces, data contracts) and validates that data passes correctly between components — without knowing each module's internal logic.

Example: Verify that when payment service returns SUCCESS, order service correctly updates order status to 'Confirmed'.

Penetration Testing

Tester knows the architecture and tech stack, enabling targeted security tests: SQL injection, XSS attacks, authentication bypasses based on known vulnerability patterns.

Example: Knowing the app uses MySQL, test: ' OR '1'='1 in login field to check for SQL injection vulnerability.

Static vs Dynamic Testing

Section 06 — Two Fundamental Testing Paradigms

Static Testing

Code is NEVER executed. Defects are found by examining artifacts — source code, design documents, requirements — manually or with automated analysis tools.

Reviews

Informal/formal examination of code or docs by peers. Goal: find defects early, share knowledge, improve quality before execution.

Walkthroughs

Author leads reviewers through document/code step-by-step. Questions and suggestions encouraged. Less formal than inspections.

Inspections

Most formal type. Defined process with roles (moderator, reader, recorder, author). Produces a defect log and follow-up actions.

Static Analysis Tools

Automated tools scan code without execution. Tools: SonarQube, Checkstyle, PMD, FindBugs. Detect: code smells, null pointer risks, duplicates.

Dynamic Testing

Code IS executed. The system is run with specific inputs and actual outputs are compared to expected results.

Unit Testing

Tests individual functions/methods in isolation. Tools: JUnit (Java), pytest (Python). Typically done by developers during coding.

Integration Testing

Tests how multiple components work together. Validates data flow between modules, APIs, and databases.

System Testing

Tests the complete, integrated system against requirements. Performed by QA teams. Covers functional and non-functional requirements.

Acceptance Testing (UAT)

Final validation by end users/stakeholders. Confirms the system meets business requirements before go-live.

Experience-Based Testing Techniques

Section 07 — Exploratory, Ad-hoc & Checklist-Based

Exploratory Testing

WHEN TO USE:

When requirements are incomplete, new features released, or time is limited.

Tester simultaneously DESIGNS and EXECUTES tests — learning about the system as they go. No predefined test script. Uses intuition, curiosity, and experience.

✓ ADVANTAGES:

+ Finds defects scripted tests miss
+ Adapts in real-time to behavior
+ Excellent for usability/UX issues
+ Fast — no upfront writing required

✗ LIMITATIONS:

− Hard to measure coverage
− Results depend on tester skill
− Difficult to reproduce exactly

Ad-hoc Testing

WHEN TO USE:

Quick sanity checks, end-of-sprint validation, or when formal docs are not yet available.

Most informal technique — no planning, no documentation, no structure. Tester performs random testing based on domain knowledge. Fast defect detection.

✓ ADVANTAGES:

+ Extremely fast to start
+ No preparation needed
+ Good for quick smoke tests

✗ LIMITATIONS:

− No repeatability
− No documentation of coverage
− Coverage is unpredictable

Checklist-Based Testing

WHEN TO USE:

Regression testing, compliance verification, or when a standard set of conditions must always be validated.

Uses a pre-defined checklist of conditions that must be verified. Derived from experience, standards, or previous defects. Less rigid than formal test cases.

✓ ADVANTAGES:

+ Consistent — same items each release
+ Quick to execute
+ Good for compliance/audits
+ Reusable across releases

✗ LIMITATIONS:

− May become outdated
− Limited to known failure modes
− Doesn't capture full detail

Risk-Based Testing

Section 08 — Prioritizing Testing Where It Matters Most

Risk-Based Testing (RBT) prioritizes testing based on RISK = Likelihood of failure × Impact of failure. High-risk areas receive the most testing effort. This ensures the most critical parts of the system are validated most thoroughly.

Risk Assessment Matrix

Likelihood ↓ / Impact →	LOW Impact	MEDIUM Impact	HIGH Impact
HIGH Likelihood	MEDIUM	HIGH	CRITICAL
MEDIUM Likelihood	LOW	MEDIUM	HIGH
LOW Likelihood	LOW	LOW	MEDIUM

Real-World Example: E-Commerce Platform

CRITICAL: Payment processing (high change likelihood + catastrophic if fails)
Test first and most exhaustively.
HIGH: User login/session management — Security risk, tested next.
MEDIUM: Product search filters — Moderate business impact.
LOW: Admin report export — Internal use, low user impact.

How to Apply Risk-Based Testing

1. **Identify Components**
   List all features, modules, and functions to be tested.
2. **Assess Likelihood**
   Rate probability of failure (1–5) based on complexity, history, and change frequency.
3. **Assess Impact**
   Rate business impact if it fails (financial, reputational, safety, legal).
4. **Calculate Risk Score**
   Multiply Likelihood × Impact. Higher score = higher testing priority.
5. **Allocate Testing Effort**
   Spend 60–70% of testing effort on high-risk areas; less on low-risk.

SECTION 09

Test Design Techniques

in Automation

Applying testing techniques using Selenium, Playwright, IntelliJ & Eclipse

Test Techniques in Automation — Selenium & Playwright

Section 09 — From Test Design to Automated Scripts

Selenium — EP & BVA Example

Scenario: Test age field (valid: 18–65) using EP and BVA

```
        .sendKeys("35"); // Valid partition
  driver.findElement(By.id("submit"))
        .click();
  Assert.assertEquals(driver
    .findElement(By.id("msg"))
    .getText(), "Age Accepted");
}
@Test
public void testBoundaryMin() {
  driver.findElement(By.id("age"))
        .sendKeys("18"); // Min boundary
  Assert.assertEquals(driver
    .findElement(By.id("msg"))
    .getText(), "Age Accepted");
}
@Test
public void testInvalidLow() {
  driver.findElement(By.id("age"))
        .sendKeys("5"); // Below valid range
  Assert.assertEquals(driver
```

Tools: Selenium WebDriver + Java + JUnit + IntelliJ IDEA
Used in: CI/CD pipelines with Jenkins/GitHub Actions

Playwright — Decision Table + State Transition

Scenario: Test login states (valid/invalid credentials)

```
import { test, expect } from '@playwright/test';

// Decision table row 1 – valid creds
test('valid login -> dashboard', async ({ page }) => {
  await page.goto('/login');
  await page.fill('#email', 'user@test.com');
  await page.fill('#password', 'SecurePass1!');
  await page.click('#loginBtn');
  await expect(page).toHaveURL('/dashboard');
});

// State transition: 3 fails -> LOCKED state
test('3 wrong passwords -> locked', async ({ page }) => {
  for (let i = 0; i < 3; i++) {
    await page.fill('#email', 'user@test.com');
    await page.fill('#password', 'wrongpass');
    await page.click('#loginBtn');
  }
  await expect(page.locator('#error'))
    .toHaveText('Account locked');
});
```

Tools: Playwright + TypeScript + VS Code / WebStorm
Advantage: Built-in auto-wait, multi-browser, mobile emulation

Development Environments — IntelliJ IDEA & Eclipse

Section 09 — Tooling for Quality Engineers

IntelliJ IDEA

by JetBrains

Industry-leading Java/Kotlin IDE. Preferred by most QA automation engineers for Selenium and Playwright Java test development.

Key Features for QA Engineers:

- Smart code completion and refactoring for test code
- Built-in JUnit & TestNG test runner with visual results
- Maven/Gradle integration for dependency management
- Git integration — commit, push, branch from IDE
- Coverage runner — highlights uncovered code lines
- Debugger — step through test execution line by line
- Plugin: Cucumber support for BDD test writing
- Remote run support — execute tests on CI servers

Eclipse IDE

by Eclipse Foundation (Open Source)

Free, open-source IDE widely used in enterprise Java development and QA automation. Highly extensible through plugins.

Key Features for QA Engineers:

- JUnit integration — run tests directly from IDE
- TestNG plugin — advanced test configuration options
- Maven plugin (m2e) — manage Selenium/Playwright deps
- EclEmma — code coverage visualization plugin
- Git support via EGit plugin
- Selenium IDE plugin — record and playback browser tests
- Step debugger — debug failing automated tests
- Free and open-source — no license cost for interns

SECTION 10

Real-World Application

Scenario

Applying all techniques to a real e-commerce login & checkout system

Real-World Scenario — E-Commerce Application

Applying Multiple Techniques to a Login & Checkout System

System: An e-commerce web application with user registration, login, product browsing, shopping cart, and payment checkout. Let's see how each testing technique contributes to finding different categories of defects.

Technique Applied	Feature Tested	Defect Found	Key Test Case
Equivalence Partitioning	Registration — Age Field	*System accepts age=0 (invalid partition not handled)*	Test with age=-1 (invalid low), age=25 (valid), age=200 (invalid high).
Boundary Value Analysis	Password Length (8–20 chars)	*System allows 7-char passwords — off-by-one error*	Test: 7 chars (fail), 8 chars (pass), 9 (pass), 20 (pass), 21 chars (fail).
Decision Table	Login — Credentials + Account Status	*Suspended accounts with correct credentials bypass suspension*	Combination: valid creds + suspended = 'Account suspended', not login.
State Transition	Login Attempt States	*Account doesn't lock after 3 failed attempts — security gap*	Fail 1 → Fail 2 → Fail 3 → Account LOCKED (no further attempts).
BVA (Checkout)	Free Shipping at $50+	*Free shipping applied at $49.99 — boundary not handled*	Test: $49.99 (no free ship), $50.00 (free ship), $50.01 (free ship).
Error Guessing	Payment Form	*SQL injection in credit card field crashes payment page*	Enter: 4111 1111 1111 1111' DROP TABLE orders;-- in card number field.

Key Insight: No single technique found ALL defects. The combination of multiple techniques achieves comprehensive quality.

Best Practices for Using Testing Techniques

Section 11 — Maximizing Quality Engineering Effectiveness

01 Combine Multiple Techniques

Never rely on just one technique. Use BVA alongside EP, add state transition for workflow, and layer in error guessing. Each catches different defect types — combination maximizes coverage.

02 Let Requirements Drive Selection

Complex conditional logic → Decision Table. Input ranges → EP + BVA. Sequential workflows → State Transition. No requirements → Exploratory. Match technique to the problem.

03 Prioritize Using Risk-Based Approach

Not all features need equal testing depth. Identify high-risk, high-impact areas first. Allocate effort proportionally — more resources on payment, authentication, data integrity.

04 Design Tests Before Writing Code (Shift Left)

Work with developers during requirements phase. Early test design catches ambiguities in requirements — far cheaper than fixing production bugs. Use static testing on specifications.

05 Translate Techniques Directly to Automation

Every technique can become automated: EP partitions → @ParameterizedTest. BVA boundaries → data-driven arrays. State transitions → Playwright scenario flows.

06 Document and Peer-Review Test Designs

Test designs (partitions, decision tables, state diagrams) should be reviewed. Mistakes in test design propagate to hundreds of test cases — catch them early.

Common Mistakes in Testing Techniques

Section 12 — What to Avoid as a QA Engineer

✗ Testing Only the Happy Path

CRITICAL

Mistake: Only testing where everything works perfectly. Most production bugs occur in error handling, edge cases, and exception flows. Always test what happens when things go wrong.

Fix: For every positive test, design at least 2 negative tests.

✗ Applying Only One Technique

HIGH

Mistake: Using only equivalence partitioning and skipping BVA, state transitions, or decision tables. Each catches a completely different class of defect — one technique gives false coverage confidence.

Fix: Combine at least 3 techniques per feature module.

✗ Ignoring Edge Cases & Boundaries

CRITICAL

Mistake: Testing with 'typical' values like 50 for an age field of 18–65. Boundary errors (off-by-one: >= vs >) are the #1 source of subtle production bugs.

Fix: Always test min, min+1, max-1, max, and just outside boundaries.

✗ Confusing EP with BVA

MEDIUM

Mistake: Treating BVA as simply 'choosing from a partition.' EP selects any representative value. BVA specifically tests boundary values. These are complementary, not the same technique.

Fix: Use EP to define partitions; use BVA to select values at partition edges.

✗ Over-Testing Low-Risk Areas

MEDIUM

Mistake: 80% of testing effort on a static 'About Us' page while complex payment processing is under-tested. This is an inefficient allocation of QA resources.

Fix: Apply risk-based testing to prioritize effort proportionally to risk.

✗ Not Reviewing Test Designs

HIGH

Mistake: Designing partitions or decision tables without peer review. An error in the test design phase propagates to all test cases built from it — creating widespread coverage gaps.

Fix: Always have a second QA engineer review all test designs before execution.

Key Takeaways

What Every QA Engineer Must Remember

1 Testing Techniques Are Not Optional

They are the scientific foundation of quality engineering. Without them, testing is guesswork. With them, it is a repeatable, measurable discipline.

2 Each Technique Targets a Specific Defect Type

EP & BVA → input validation. Decision Tables → logic combinations. State Transition → workflow bugs. Use Case → user journey gaps. Coverage → code logic.

3 Combine Techniques for Maximum Coverage

No single technique provides complete assurance. Layer multiple techniques to validate from multiple perspectives simultaneously.

4 Automation Amplifies Technique Value

Every technique can be automated using Selenium, Playwright, or JUnit — making them repeatable, scalable, and part of your CI/CD pipeline.

5 Risk Dictates Priority

Not everything can be tested equally. Use risk-based testing to focus your most intensive technique application on features that matter most.

6 Quality Engineering Is a Mindset

Techniques are tools. The real skill is knowing WHEN, WHERE, and HOW to apply them. That judgment comes from practice, review, and continuous learning.